03 — Object-Oriented Programming

If you know Java, most of this will feel familiar. The main differences are syntax: `$this->`, `->` instead of `.`, `::` for static members.

Classes and objects

```php
class Product
{
    // Properties (attributes)
    public string $name;
    public float $price;

    // Constructor: called by "new"
    public function __construct(string $name, float $price)
    {
        $this->name = $name;
        $this->price = $price;
    }

    // Method
    public function priceWithTax(float $rate = 0.2): float
    {
        return round($this->price * (1 + $rate), 2);
    }
}

$keyboard = new Product('Keyboard', 49.90);   // create an object (instance)
echo $keyboard->name;                         // access a property
echo $keyboard->priceWithTax();               // call a method
```

- `$this` is the current object (like `this` in Java, `self` in Python).
- `->` accesses properties and methods of an object.
- Objects are passed by **handle**: if a function modifies an object it receives, the caller sees the change. Use `clone $obj` to get a copy.

Constructor property promotion (PHP 8)

The constructor above can be written in one go: declaring visibility in the parameters creates and assigns the properties.

```php
class Product
{
    public function __construct(
        public string $name,
        public float $price,
    ) {}
}
```

Encapsulation: visibility

Keyword	Accessible from
`public`	anywhere
`protected`	the class and its children
`private`	the class only

Good practice: keep properties `private`, expose what's needed through methods. This lets the class protect its own rules.

```php
class BankAccount
{
    private float $balance = 0;

    public function deposit(float $amount): void
    {
        if ($amount <= 0) {
            throw new InvalidArgumentException('Amount must be positive');
        }
        $this->balance += $amount;
    }

    public function getBalance(): float
    {
        return $this->balance;
    }
}

$account = new BankAccount();
$account->deposit(100);
echo $account->getBalance();   // 100
// $account->balance = 1000000;   Error: Cannot access private property
```

`readonly` properties (PHP 8.1) can be set once, in the constructor, then never changed:

```php
public function __construct(public readonly string $id) {}
```

Static members and constants

Static members belong to the **class**, not to an object. Access them with `::`.

```php
class Counter
{
    public const MAX = 10;
    private static int $count = 0;

    public static function increment(): int
    {
        return ++self::$count;    // self:: = this class
    }
}

echo Counter::MAX;
Counter::increment();
```

Inheritance

```php
class Animal
{
    public function __construct(protected string $name) {}

    public function speak(): string
    {
        return "{$this->name} makes a sound";
    }
}

class Dog extends Animal
{
    public function speak(): string              // override
    {
        return "{$this->name} barks";
    }

    public function fetch(): string
    {
        return "{$this->name} fetches the ball";
    }
}

$dog = new Dog('Rex');
echo $dog->speak();     // Rex barks
```

- A class extends **one** parent only (no multiple inheritance).
- `parent::method()` calls the parent's version, often used in constructors: `parent::__construct($name);`
- `final` on a class prevents extending it; on a method, prevents overriding it.
- `instanceof` tests the type: `$dog instanceof Animal` → `true`.

Abstract classes

An abstract class can't be instantiated; it defines a common base and forces children to implement some methods.

```php
abstract class Shape
{
    abstract public function area(): float;

    public function describe(): string
    {
        return static::class . ' with area ' . round($this->area(), 2);
    }
}
```

Polymorphism

Different classes, same method name, each with its own behavior. Code using them doesn't need to know which one it has.

```php
$animals = [new Dog('Rex'), new Cat('Felix'), new Animal('Generic')];
foreach ($animals as $animal) {
    echo $animal->speak() . PHP_EOL;   // each one answers in its own way
}
```

Interfaces

An interface is a contract: a list of methods a class promises to implement. A class can implement **several** interfaces.

```php
interface Payable
{
    public function pay(float $amount): string;
}

class CreditCard implements Payable
{
    public function pay(float $amount): string
    {
        return "Paid $amount € by card";
    }
}

class PayPal implements Payable
{
    public function pay(float $amount): string
    {
        return "Paid $amount € with PayPal";
    }
}

function checkout(Payable $method, float $total): void   // type = the interface
{
    echo $method->pay($total);
}
```

Abstract class or interface? Interface to describe **what an object can do**; abstract class to **share code** between related classes.

Traits (`trait` + `use`) let you copy methods into several unrelated classes. Good to recognize; use sparingly.

Exceptions

```php
try {
    $account->deposit(-5);
} catch (InvalidArgumentException $e) {
    echo 'Error: ' . $e->getMessage();
} finally {
    echo 'Always executed';
}
```

You can create your own: `class InsufficientFundsException extends Exception {}`.

Namespaces and autoloading

Namespaces avoid name collisions between your classes and libraries (two `User` classes can coexist), like Java packages.

```php
// src/Shop/Product.php
<?php
namespace App\Shop;

class Product { /* ... */ }
```

```php
// index.php
<?php
require __DIR__ . '/src/Shop/Product.php';

use App\Shop\Product;          // import, then use the short name

$p = new Product('Mouse', 19.5);
// or without "use": new \App\Shop\Product(...)
```

Built-in classes live in the global namespace: inside a namespace, write `new \DateTime()` (or `use DateTime;`).

Autoloading

Writing one `require` per class quickly becomes painful. An autoloader loads a class file automatically the first time the class is used:

```php
spl_autoload_register(function (string $class) {
    // App\Shop\Product -> src/Shop/Product.php
    $relative = str_replace(['App\\', '\\'], ['', '/'], $class);
    require __DIR__ . '/src/' . $relative . '.php';
});
```

In real projects, **Composer** generates this for you (PSR-4 standard). That's how Symfony, Laravel and modern WordPress plugins load their classes.

Now do the exercises.

03 — Exercises

Command-line scripts: `php file.php`.

3.1 — Bank account

Create a `BankAccount` class:

- private property `$balance`, a public readonly property `$owner` (set in the constructor)
- `deposit(float $amount)`: refuses amounts <= 0 (throw `InvalidArgumentException`)
- `withdraw(float $amount)`: refuses if the balance would go below zero (throw an exception)
- `getBalance(): float`

Test it: create an account, make a few operations, then try an invalid withdrawal inside a `try/catch` and display the error message.

3.2 — Shapes (inheritance and polymorphism)

1. Create an abstract class `Shape` with an abstract method `area(): float` and a concrete method `describe(): string` returning e.g. `Circle with area 78.54`.
2. Create `Circle` (radius), `Rectangle` (width, height) and `Square` (side). `Square` should **extend** `Rectangle`.
3. Put one of each in an array, loop over it and display each `describe()`.
4. Sort the array by area with `usort` and display again.

Hint: `M_PI` is π. `static::class` gives the real class name of the object.

3.3 — Notifications (interfaces)

1. Create an interface `Notifier` with a method `send(string $to, string $message): string`.
2. Implement it in `EmailNotifier` and `SmsNotifier` (they just return a text like `[EMAIL to bob@mail.com] Hello`).
3. `SmsNotifier` should cut messages longer than 20 characters and add `...`.
4. Write a function `notifyAll(array $notifiers, string $to, string $message)` that sends with each notifier.

3.4 — Namespaces and autoload

Reorganize exercise 3.3 in a folder:

```
3.4-namespaces/
├── index.php
└── src/
    └── Notification/
        ├── Notifier.php
        ├── EmailNotifier.php
        └── SmsNotifier.php
```

- Each class in namespace `App\Notification`.
- `index.php` registers an autoloader with `spl_autoload_register`, imports the classes with `use`, and runs the same test as 3.3.
- No `require` of the class files themselves.